

Q1. Simulate an expert system rule (age greater than 18) in Java if-else.

An expert system rule such as "if a person's age is greater than 18, they are an adult" can be simulated in Java using a simple conditional check, mimicking the fact-and-rule structure used in logic programming.

```
public class ExpertRuleDemo {  
    public static void main(String[] args) {  
        int age = 20;  
  
        // Rule: IF age > 18 THEN person is an adult  
        if (age > 18) {  
            System.out.println("Eligible: Adult");  
        } else {  
            System.out.println("Not Eligible: Minor");  
        }  
    }  
}
```

Although Java itself is not a logic-programming language, this if-else structure plays the same role as a rule in an expert system: given a fact (age = 20), it applies a rule (age > 18 implies adult) to reach a conclusion.

Q2. Convert decision rules into Java conditions.

Consider a small set of decision rules for classifying a loan applicant:

- Rule 1: IF income is greater than or equal to 50000 AND credit score is greater than or equal to 700, THEN approve the loan.
- Rule 2: IF income is greater than or equal to 50000 AND credit score is less than 700, THEN review the loan manually.
- Rule 3: IF income is less than 50000, THEN reject the loan.

```
public class LoanRulesDemo {  
    public static void main(String[] args) {  
        double income = 60000;  
        int creditScore = 680;  
  
        if (income >= 50000 && creditScore >= 700) {  
            System.out.println("Loan Approved");  
        } else if (income >= 50000 && creditScore < 700) {
```

```

        System.out.println("Manual Review Required");
    } else {
        System.out.println("Loan Rejected");
    }
}
}

```

Each natural-language decision rule maps directly onto one branch of the if-else-if chain, with the rule's condition becoming the Java boolean expression and the rule's conclusion becoming the corresponding printed outcome.

Q3. Design a Java quiz app that uses rule-based logic.

A simple quiz application can apply rule-based logic to grade a user's answers and determine a result category, similar to how an expert system reaches a conclusion from a set of facts.

```

import java.util.Scanner;

public class QuizApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int score = 0;

        System.out.println("Q1: What is the output of 5 + 3 in Java? ");
        System.out.println("a) 53 b) 8 c) Error");
        String ans1 = sc.next();
        if (ans1.equalsIgnoreCase("b")) score++;

        System.out.println("Q2: Which keyword is used for inheritance? ");
        System.out.println("a) implements b) extends c) inherits");
        String ans2 = sc.next();
        if (ans2.equalsIgnoreCase("b")) score++;

        // Rule-based result classification
        if (score == 2) {
            System.out.println("Result: Excellent");
        } else if (score == 1) {
            System.out.println("Result: Good, but review the concepts");
        } else {
            System.out.println("Result: Needs Improvement");
        }
    }
}

```

The application collects facts (the user's answers), evaluates them against known correct answers to build a score, and then applies a small set of rules (based on the resulting score) to classify the outcome -- the same rule-based pattern used by simple expert systems.